

SECTION 10 — JAVA CODING AND DSA FOR SDET (Senior SDET Masterclass)

Topics Covered

- 10.1 Problem-Solving Protocol (The C-E-B-O-T Senior Coding Framework)
- 10.2 Arrays (Two Pointers, In-Place Reversals, & Merge Intervals)
- 10.3 Strings (Character Arrays, In-Place Manipulations, & Palindromes)
- 10.4 Frequency Counting (Array Hash Maps vs. Hash Table Overhead)
- 10.5 HashMap & HashSet Algorithmic Patterns (Two Sum & Group Anagrams)
- 10.6 Two Pointers Technique (Opposite-Direction vs. Fast-and-Slow Pointers)
- 10.7 Sliding Window (Fixed Window vs. Dynamic Expanding/Shrinking Window)
- 10.8 Prefix Sum (Range Queries & Subarray Sum Optimization)
- 10.9 Monotonic Stack (Balanced Parentheses & Next Greater Element)
- 10.10 Queue and Deque (Sliding Window Maximum & BFS DOM Traversals)
- 10.11 Binary Search (`mid = lo + (hi - lo) / 2` & Rotated Sorted Arrays)
- 10.12 Linked Lists (In-Place Reversal & Floyd's Cycle Detection)
- 10.13 Recursion & Backtracking (Base Conditions & Call Stack Frames)
- 10.14 Trees (DOM Representation, DFS vs. BFS, & Lowest Common Ancestor)
- 10.15 Sorting Algorithms & Custom Comparators (`thenComparing` Chaining)
- 10.16 Big-O Time & Space Complexity Master Reference
- 10.17 Edge Case Taxonomy (The 8 Mandatory Guardrails)
- 10.18 Senior Code Review Standards for SDET Coding Rounds
- 10.19 Top 5 SDET-Specific Algorithmic Problems
- 10.20 Production Engine: Algorithmic Log Parsing & Sliding Window Error Burst Detection
- 10.21 Production Engine: Deep Payload Structural Validation & Recursive JSON Diffing

10.1 Problem-Solving Protocol (The C-E-B-O-T Framework)

In Senior SDET coding rounds (Deloitte, FAANG, tier-1 tech), interviewers do not just evaluate if your code compiles; they evaluate your **engineering communication, edge-case vigilance, and trade-off analysis**.

C-E-B-O-T CODING PROTOCOL	
1. [C] CLARIFY	Restate problem, identify input sizes, confirm nullability, case-sensitivity, and ordering.
2. [E] EDGES	List 4+ edge cases explicitly before writing a line.
3. [B] BRUTE	State naive $O(N^2)$ solution out loud with Big-O.
4. [O] OPTIMIZE	Explain how a data structure trades space for time.

5. [T] TEST	Trace sample input with pointers before saying done.
-------------	--

10.2 Arrays & Two Pointers: Merge Intervals

Problem: Merge Overlapping Test Execution Time Windows

Given an array of test execution intervals `intervals[i] = [start_i, end_i]`, merge all overlapping intervals.

```
package com.deloitte.sdet.dsa;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.Comparator;
import java.util.List;

public final class IntervalMerger {

    public static int[][] mergeIntervals(int[][] intervals) {
        if (intervals == null || intervals.length <= 1) {
            return intervals;
        }

        // Sort intervals based on starting timestamp O(N log N)
        Arrays.sort(intervals, Comparator.comparingInt(a -> a[0]));

        List<int[]> merged = new ArrayList<>();
        int[] current = intervals[0];
        merged.add(current);

        for (int i = 1; i < intervals.length; i++) {
            int currentEnd = current[1];
            int nextStart = intervals[i][0];
            int nextEnd = intervals[i][1];

            if (currentEnd >= nextStart) {
                // Overlap detected: expand the current window
                current[1] = Math.max(currentEnd, nextEnd);
            } else {
                // Disjoint interval: move to next
                current = intervals[i];
                merged.add(current);
            }
        }

        return merged.toArray(new int[merged.size()][2]);
    }
}
```

- **Time Complexity:** $O(N \log N)$ due to initial array sort.
- **Space Complexity:** $O(N)$ to hold merged intervals.

10.4 Frequency Counting: Anagram Validation & Array Hash Maps

The Senior Trick: `int[26]` / `int[128]` Beats `HashMap<Character, Integer>`

Creating a `HashMap<Character, Integer>` allocates hundreds of `Node` and `Character` wrapper objects on the heap. An integer array operates strictly in CPU cache with zero garbage collection overhead.

```
public static boolean isAnagram(String s1, String s2) {
    if (s1 == null || s2 == null || s1.length() != s2.length()) {
        return false;
    }

    int[] frequencies = new int[26];
    for (int i = 0; i < s1.length(); i++) {
        frequencies[s1.charAt(i) - 'a']++;
        frequencies[s2.charAt(i) - 'a']--;
    }

    for (int count : frequencies) {
        if (count != 0) return false;
    }
    return true; // O(N) Time | O(1) Auxiliary Space
}
```

10.5 HashMap Patterns: Two Sum

Given an array of HTTP response latencies and a target timeout threshold, return indices of two requests whose sum equals target.

```
public static int[] twoSum(int[] nums, int target) {
    if (nums == null || nums.length < 2) {
        return new int[0];
    }

    // Map: Key = Complement needed, Value = Index
    Map<Integer, Integer> complementMap = new HashMap<>();

    for (int i = 0; i < nums.length; i++) {
        int complement = target - nums[i];
        if (complementMap.containsKey(complement)) {
            return new int[]{complementMap.get(complement), i};
        }
        complementMap.put(nums[i], i);
    }

    return new int[0]; // O(N) Time | O(N) Space
}
```

10.7 Sliding Window: Longest Substring Without Repeating Characters

```
public static int lengthOfLongestSubstring(String s) {
    if (s == null || s.isEmpty()) return 0;

    int maxLength = 0;
    int left = 0;
    // Map stores character and its last seen index
    Map<Character, Integer> lastSeen = new HashMap<>();

    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);
        if (lastSeen.containsKey(c)) {
            // Move left pointer past the previous occurrence
            left = Math.max(left, lastSeen.get(c) + 1);
        }
        lastSeen.put(c, right);
        maxLength = Math.max(maxLength, right - left + 1);
    }

    return maxLength; // O(N) Time | O(min(N, AlphabetSize)) Space
}
```

10.9 Stack: Valid Parentheses (DOM Tag Validation)

```
public static boolean isValidDomStructure(String s) {
    if (s == null || s.length() % 2 != 0) return false;

    Deque<Character> stack = new ArrayDeque<>();
    for (char c : s.toCharArray()) {
        if (c == '(') stack.push('(');
        else if (c == '{') stack.push('{');
        else if (c == '[') stack.push('[');
        else {
            if (stack.isEmpty() || stack.pop() != c) {
                return false;
            }
        }
    }
    return stack.isEmpty(); // O(N) Time | O(N) Space
}
```

10.11 Binary Search: Search in Rotated Sorted Array

```
public static int searchRotatedArray(int[] nums, int target) {
    if (nums == null || nums.length == 0) return -1;

    int lo = 0;
    int hi = nums.length - 1;
```

```

while (lo <= hi) {
    int mid = lo + (hi - lo) / 2; // Prevents 32-bit integer overflow!

    if (nums[mid] == target) return mid;

    // Determine which half is normally ordered
    if (nums[lo] <= nums[mid]) {
        // Left half is sorted
        if (target >= nums[lo] && target < nums[mid]) {
            hi = mid - 1;
        } else {
            lo = mid + 1;
        }
    } else {
        // Right half is sorted
        if (target > nums[mid] && target <= nums[hi]) {
            lo = mid + 1;
        } else {
            hi = mid - 1;
        }
    }
}
return -1; // O(log N) Time | O(1) Space
}

```

10.12 Linked Lists: Floyd's Cycle Detection Algorithm

Detect if a test execution dependency graph contains an infinite cycle:

```

public static boolean hasCycle(ListNode head) {
    if (head == null || head.next == null) return false;

    ListNode slow = head;
    ListNode fast = head.next;

    while (slow != fast) {
        if (fast == null || fast.next == null) {
            return false; // Reached end of list -> No cycle
        }
        slow = slow.next;          // 1 step
        fast = fast.next.next;     // 2 steps
    }
    return true; // Fast met slow -> Cycle confirmed! O(N) Time | O(1) Space
}

```

10.20 Algorithmic Log Parsing & Sliding Window Error Burst Engine

High-Stakes Senior SDET Problem

In a 5,000+ nightly test run, multiple gigabytes of logs are generated.

Write an engine that:

1. Streams gigabyte log files line-by-line without loading the entire file into memory (preventing `OutOfMemoryError`).
2. Extracts ISO timestamps and error codes using compiled regex.
3. Uses a **Sliding Window** to detect if >10 errors occurred within any rolling 5-minute window (detecting infrastructure cascading failures).

```
package com.deloitte.sdet.engine;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.time.Instant;
import java.util.*;
import java.util.regex.Matcher;
import java.util.regex.Pattern;
import java.util.stream.Stream;

public final class LogAnalyticsEngine {

    // Pre-compiled regex with named capture groups
    private static final Pattern LOG_PATTERN = Pattern.compile(
        "^(?<ts>\\d{4}-\\d{2}-\\d{2}T\\S+)\\s+\\[\\((?<lvl>ERROR|WARN|INFO)\\)\\]\\s+\\[\\((?<code>[A-Z0-9-])\\)\\]";

    public static List<String> detectErrorBursts(Path logFile, int threshold, long windowSeconds) {
        List<Long> errorTimestamps = new ArrayList<>();

        // Stream lines lazily: Constant O(1) JVM Heap Memory footprint
        try (Stream<String> lines = Files.lines(logFile)) {
            lines.forEach(line -> {
                Matcher matcher = LOG_PATTERN.matcher(line);
                if (matcher.matches() && "ERROR".equals(matcher.group("lvl"))) {
                    long epoch = Instant.parse(matcher.group("ts")).getEpochSecond();
                    errorTimestamps.add(epoch);
                }
            });
        }

        // Sliding Window across timestamps
        List<String> burstAlerts = new ArrayList<>();
        int left = 0;

        for (int right = 0; right < errorTimestamps.size(); right++) {
            while (errorTimestamps.get(right) - errorTimestamps.get(left) > windowSeconds) {
                left++;
            }
            int errorsInWindow = right - left + 1;
            if (errorsInWindow >= threshold) {
                burstAlerts.add(String.format("BURST DETECTED at epoch %d: %d errors in %d seconds",
                    errorTimestamps.get(right), errorsInWindow, windowSeconds));
                // Slide window forward to avoid duplicate alerts for the same burst
                left = right + 1;
            }
        }
    }
}
```

```

        }
    }

    return burstAlerts;
}
}

```

10.21 Recursive Nested JSON Structural Diffing Engine

High-Stakes Senior SDET Problem

When validating complex microservice responses, API contracts drift silently. Write a recursive diffing engine that compares two arbitrary nested JSON structures (represented as `Map<String, Object>` and `List<Object>`), detecting:

- Missing keys
- Unexpected keys
- Type mismatches
- Value differences (with capability to ignore dynamic fields like timestamps/UUIDs).

```

package com.deloitte.sdet.engine;

import java.util.*;

public final class JsonStructuralDiffer {

    public static List<String> diff(Map<String, Object> expected, Map<String, Object> actual, Set<String> ignoredKeys, List<String> discrepancies) {
        List<String> discrepancies = new ArrayList<>();
        compareMaps("", expected, actual, ignoredKeys, discrepancies);
        return discrepancies;
    }

    @SuppressWarnings("unchecked")
    private static void compareMaps(String path, Map<String, Object> exp, Map<String, Object> act, Set<String> ignored, List<String> diffs) {
        // 1. Check for missing keys or mismatched values
        for (String key : exp.keySet()) {
            if (ignored.contains(key)) continue;
            String currentPath = path.isEmpty() ? key : path + "." + key;

            if (!act.containsKey(key)) {
                diffs.add("MISSING_KEY: Expected key missing at [" + currentPath + "]");
                continue;
            }

            Object expVal = exp.get(key);
            Object actVal = act.get(key);

            if (expVal instanceof Map && actVal instanceof Map) {
                compareMaps(currentPath, (Map<String, Object>) expVal, (Map<String, Object>) actVal, ignored, diffs);
            } else if (expVal instanceof List && actVal instanceof List) {
                compareLists(currentPath, (List<Object>) expVal, (List<Object>) actVal, ignored, diffs);
            }
        }
    }
}

```

```

        } else if (!Objects.equals(expVal, actVal)) {
            diffs.add(String.format("VALUE_MISMATCH at [%s]: expected='%s', actual='%s'", cu
        }
    }

    // 2. Check for unexpected keys in actual
    for (String key : act.keySet()) {
        if (ignored.contains(key)) continue;
        String currentPath = path.isEmpty() ? key : path + "." + key;
        if (!exp.containsKey(key)) {
            diffs.add("UNEXPECTED_KEY: Found extra key at [" + currentPath + "]");
        }
    }
}

@SuppressWarnings("unchecked")
private static void compareLists(String path, List<Object> exp, List<Object> act,
                                Set<String> ignored, List<String> diffs) {
    if (exp.size() != act.size()) {
        diffs.add(String.format("LIST_SIZE_MISMATCH at [%s]: expected=%d, actual=%d", path,
        return;
    }
    for (int i = 0; i < exp.size(); i++) {
        String currentPath = path + "[" + i + "]";
        Object expItem = exp.get(i);
        Object actItem = act.get(i);
        if (expItem instanceof Map && actItem instanceof Map) {
            compareMaps(currentPath, (Map<String, Object>) expItem, (Map<String, Object>) ac
        } else if (!Objects.equals(expItem, actItem)) {
            diffs.add(String.format("LIST_ITEM_MISMATCH at [%s]: expected='%s', actual='%s'"
        }
    }
}
}
}
}

```